

Actividad 1: Fundamentos de APIs y Estructuras JSON en Redes

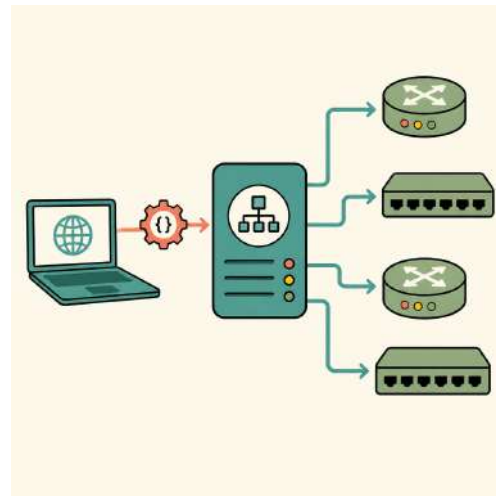
Conceptos Clave

API (Application Programming Interface): Un contrato que define cómo un software solicita servicios a otro.

Controlador de Red: Un servidor centralizado (como Cisco DNA Center o APIC) que gestiona dispositivos de red mediante APIs.

JSON (JavaScript Object Notation): Formato ligero de intercambio de datos basado en pares clave-valor.

En las redes modernas definidas por software (SDN), la interacción entre el administrador y el hardware ha cambiado. Ya no dependemos únicamente de la línea de comandos (CLI); ahora utilizamos el modelo **cliente-servidor** a través de APIs. En este contexto, el **cliente** suele ser un script de Python o una plataforma de gestión, mientras que el **servidor** es el controlador de red (como APIC en entornos ACI o DNA Center en redes empresariales).



Interacción entre scripts y controladores de red.

El controlador actúa como un intermediario que expone una API. Cuando el cliente envía una solicitud para consultar el estado de una interfaz, el controlador responde con datos estructurados, generalmente en formato **JSON**. Este formato es esencial porque permite que las máquinas "entiendan" la topología y el estado de la red de forma jerárquica y estandarizada, facilitando la automatización de tareas complejas que antes requerían intervención manual en cada router o switch.

Parte 1: Análisis de Estructura JSON

Observa el siguiente bloque de código que representa el estado de las interfaces en un router de núcleo (Core Router). Utiliza esta estructura para responder a las preguntas a continuación.

```
{
  "hostname": "R1-Core",
  "uptime_days": 45,
  "managed_by_apic": true,
  "interfaces": [
    {
      "name": "GigabitEthernet0/0",
      "description": "Link to WAN",
      "enabled": true,
      "mtu": 1500
    },
    {
      "name": "GigabitEthernet0/1",
      "description": null,
      "enabled": false,
      "mtu": 1500
    }
  ]
}
```

[illegible]

Parte 2: Detección de Errores de Sintaxis

En la automatización de redes, un error de sintaxis en un payload JSON puede causar que la configuración de un dispositivo falle. Analiza el siguiente fragmento de código destinado a configurar una VLAN y responde.

```
{  
  "vlan_id": 10  
  "name": "Usuarios_Ventas"  
  "tagged": true,  
  "description": "VLAN para el departamento de ventas",  
}
```

3. ¿Qué errores de sintaxis impiden que este JSON sea válido según los estándares de red y programación?

Selecciona las dos opciones correctas.

- A) El valor de 'vlan_id' debería estar entre comillas dobles obligatoriamente.
- B) Las claves como 'vlan_id' no deben llevar guiones bajos.
- C) Existe una coma final (trailing comma) después del valor de 'description'.
- D) Faltan comas para separar los pares clave-valor después de 'vlan_id' y 'name'.

4. Reflexión Técnica: Explica por qué es crítico que un controlador como Cisco DNA Center reciba un JSON perfectamente validado antes de aplicar cambios en la red.

Actividad 2: Análisis de Datos de Red con Python

Conceptos Clave

json.loads(): Función del módulo `json` de Python que convierte una cadena de texto en formato JSON a un diccionario o lista de Python.

Parsing: El proceso de analizar una estructura de datos para extraer información específica.

Parte 1: Análisis de Inventario

Imagina que estás automatizando la auditoría de una red empresarial. Has recibido el siguiente objeto JSON que representa el inventario actual de dispositivos. Tu objetivo es identificar qué equipos están fuera de línea (down) para generar un reporte de incidentes.

```
[
{"hostname": "Core-SW01", "ip": "10.0.0.1", "status": "up"},
{"hostname": "Dist-SW02", "ip": "10.0.0.2", "status": "down"},
{"hostname": "Access-SW03", "ip": "10.0.0.3", "status": "up"},
{"hostname": "Edge-R01", "ip": "172.16.1.1", "status": "down"}
]
```

1. Escribe el pseudocódigo o completa el script de Python necesario para procesar la variable `inventario_json` (que contiene el texto anterior). El script debe filtrar e imprimir únicamente los **hostnames** de los dispositivos cuyo estado sea `'down'`.

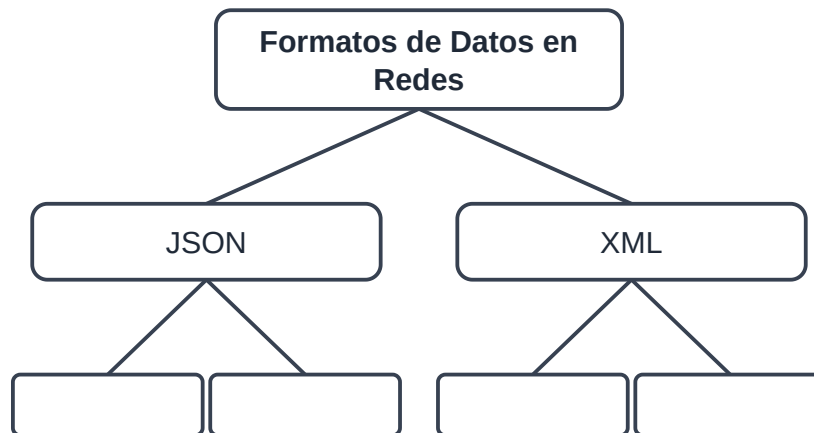
Parte 2: Comparativa de Formatos (JSON vs. XML)

En redes modernas, REST APIs suelen preferir JSON por ser más ligero, pero muchos sistemas heredados o protocolos como NETCONF utilizan XML. Observa cómo se estructuran los mismos datos en ambos formatos.

Formato JSON	Formato XML
<pre>{ "hostname": "R1", "status": "up" }</pre>	<pre>xml <device> <hostname>R1</hostname> <status>up</status> </device></pre>

2. Utiliza el siguiente organizador gráfico para clasificar las características de JSON y XML basándote en la comparativa anterior y el contenido del Tema 4.

Comparativa JSON vs XML



3. ¿Cuál es una diferencia fundamental en el manejo de tipos de datos entre JSON y XML según el material de estudio?

- A) JSON tiene tipos de datos nativos (números, booleanos), mientras que en XML todo se trata inicialmente como texto.
- B) XML es más ligero que JSON porque no utiliza comillas dobles en sus valores.
- C) JSON requiere etiquetas de cierre para cada par clave-valor, lo que aumenta su tamaño.

4. Explica por qué un ingeniero de redes preferiría usar JSON sobre XML al trabajar con una API REST moderna para gestionar miles de dispositivos simultáneamente.

.....

.....

.....

.....

.....

.....

.....

.....

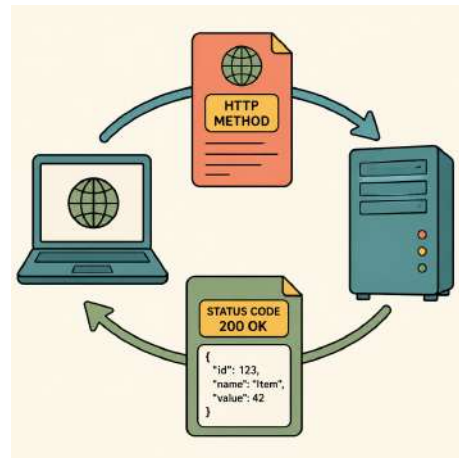
Interacción con REST APIs y Protocolo HTTP

Conceptos Clave

REST (Representational State Transfer) — Estilo arquitectónico para sistemas distribuidos que utiliza los métodos estándar de HTTP para manipular recursos identificados por URIs.

Stateless (Sin estado) — Cada petición del cliente al servidor debe contener toda la información necesaria para entenderla, sin depender de sesiones previas.

En la automatización de redes modernas, el protocolo HTTP es el lenguaje fundamental para interactuar con controladores como Cisco DNA Center o interfaces RESTCONF. Los **métodos HTTP** definen la intención de la operación: mientras que un método `GET` se limita a recuperar información del estado de una interfaz, un método `POST` o `PUT` implica una alteración en la configuración del dispositivo.



Interacción Cliente-Servidor en APIs

Para que esta comunicación sea efectiva, el servidor responde con **códigos de estado**. Estos códigos son números de tres dígitos que indican si una petición se completó con éxito (2xx), si hubo un error por parte del cliente (4xx) como falta de permisos o rutas inexistentes, o si el servidor falló internamente (5xx).

Comprender esta relación es vital para el diagnóstico de fallas en scripts de infraestructura como código (IaC).

Parte 1: Métodos HTTP y Gestión de Recursos

Relaciona cada acción de administración de red con el método HTTP más apropiado. Escribe el método (GET, POST, PUT, PATCH, DELETE) en la columna correspondiente.

Acción sobre el Recurso de Red	Método HTTP
Obtener la lista de todas las VLANs configuradas en un switch.	

Crear un nuevo objeto de red (Network Object) en un firewall.	
Reemplazar la configuración completa de una interfaz OSPF.	
Eliminar una ruta estática que ya no es necesaria.	
Modificar únicamente la descripción de una interfaz (cambio parcial).	

Parte 2: Análisis de Códigos de Estado

Como ingeniero de redes, estás auditando logs de una API RESTCONF. Predice qué código de estado devolvería el controlador en las siguientes situaciones.

1. Un script intenta consultar el estado de la interfaz `GigabitEthernet99`, pero dicha interfaz no existe físicamente ni lógicamente en el router. ¿Qué código devuelve la API?

- A) 200 OK
- B) 404 Not Found
- C) 500 Internal Server Error
- D) 201 Created

2. Se envía una petición `POST` para configurar un nuevo túnel VPN, pero el token de autenticación en el encabezado `Authorization` ha expirado o es inválido.

- A) 401 Unauthorized
- B) 204 No Content
- C) 400 Bad Request

Parte 3: Diagnóstico y Justificación

3. Un administrador de red intenta actualizar la IP de un servidor mediante un método `PUT`. La API responde con un código **400 Bad Request**. Explica dos posibles razones técnicas relacionadas con la estructura de la petición (payload) que podrían causar este error específico.

4. ¿Cuál es la diferencia principal entre el uso de `POST` y `PUT` al configurar una interfaz de red, considerando la propiedad de **idempotencia**?

Automatización de Pruebas en Postman

Conceptos de Automatización

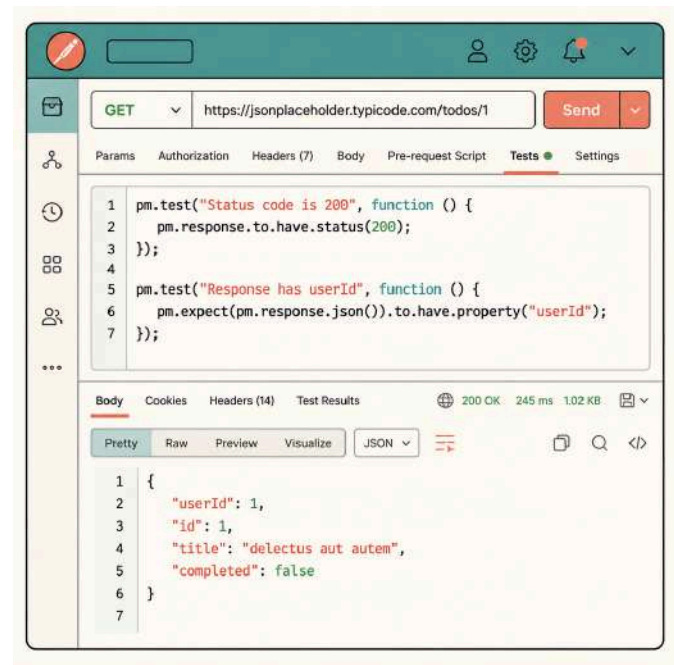
pm.test() — Función de Postman para definir una prueba. Recibe un nombre y una función de callback.

pm.expect() — Librería de aserciones para validar que los datos recibidos coinciden con lo esperado.

pm.response — Objeto que contiene toda la información de la respuesta del servidor (cuerpo, tiempo, código).

En la gestión de redes modernas, no basta con enviar una petición y observar el resultado manualmente. La automatización exige que el propio cliente (Postman) valide si la respuesta es correcta. Las pruebas en Postman se escriben en JavaScript dentro de la pestaña **Tests**.

Estas pruebas permiten verificar criterios críticos de rendimiento y seguridad. Por ejemplo, en una infraestructura de misión crítica, un tiempo de respuesta elevado puede indicar congestión en el controlador de red. Del mismo modo, validar la presencia de campos específicos como `hostname` asegura que el inventario de red está entregando la información completa necesaria para otros scripts de automatización.



Pestaña de pruebas en Postman

Parte 1: Análisis de Scripts de Prueba

Observa los siguientes ejemplos de scripts utilizados para validar respuestas de una API de red:

Objetivo de la Prueba	Código del Script (JavaScript)
Validar código de estado 200	<pre>pm.test("Estado 200", () => { pm.response.to.have.status(200); });</pre>

Verificar formato JSON	<pre>pm.test("Cuerpo es JSON", () => { pm.response.to.be.withBody; pm.response.to.be.json; });</pre>
------------------------	---

1. Si una API de red tarda 500ms en responder y tienes configurada una prueba para validar que el tiempo sea menor a 200ms, ¿cuál será el resultado en el panel de Postman?

- A) La prueba fallará (FAIL) y mostrará un error de aserción.
- B) La petición será cancelada automáticamente por Postman.
- C) La prueba pasará (PASS) porque el código de estado sigue siendo 200.

Parte 2: Desarrollo de Pruebas Personalizadas

2. Basándote en los ejemplos anteriores, escribe el script de prueba necesario para verificar que el **tiempo de respuesta sea menor a 200ms**. Utiliza la sintaxis `pm.response.responseTime`.

```
.....  
.....  
.....  
.....  
.....  
.....  
.....  
.....
```

3. Escribe un script que valide que el cuerpo de la respuesta (JSON) contenga el campo `hostname`.

Pista: Primero debes obtener el JSON con `const data = pm.response.json();` y luego usar `pm.expect(data).to.have.property('...')`.

```
.....  
.....  
.....  
.....  
.....  
.....  
.....  
.....
```

Parte 3: Aplicación en Escenarios de Red

4. Imagina que estás automatizando la verificación de un router tras una actualización. La API devuelve el siguiente JSON:

```
{  
  "hostname": "R1-Core",  
  "status": "up",  
  "uptime": "15d 4h"  
}
```

Explica por qué es fundamental validar tanto el **código de estado HTTP** como el **contenido del JSON** antes de marcar una tarea de automatización como exitosa.

Desarrollo de Microservicios con Flask y Requests

Conceptos Clave

Flask: Micro-framework de Python para crear servidores web y APIs de forma ligera.

Requests: Librería cliente para enviar peticiones HTTP (GET, POST, etc.) a un servidor.

Manejo de Excepciones: Técnica para capturar errores (como fallos de conexión) y evitar que el programa se detenga bruscamente.

Parte 1: El Servidor (Flask)

Imagina que estás construyendo un sistema de inventario para los routers de tu red. Debes completar el código del servidor para que pueda recibir nuevos dispositivos y listar los existentes.

Instrucciones: Completa los espacios en blanco en las rutas de Flask para manejar la lógica de red.

Parte 2: El Cliente (Requests)

Ahora, actúa como el administrador de red que necesita registrar un nuevo router en el servidor que acabas de crear. Debes enviar un objeto JSON con los datos del dispositivo y manejar posibles errores de red.

Tarea de Programación	Escribe el código en el espacio de la derecha
- Define un diccionario llamado <code>nuevo_router</code> con <code>hostname: 'R1-Edge'</code> e <code>ip: '10.0.0.1'</code>. - Envía una petición POST a <code>http://localhost:5000/devic</code> es usando el parámetro <code>json</code>. - Implementa un bloque <code>try/except</code> para capturar <code>requests.exceptions.ConnectionError</code>.	

Análisis Técnico: ¿Por qué es crítico utilizar un bloque `try/except` específicamente con `ConnectionError` al automatizar tareas de red con Python?

- A) Porque Flask no permite conexiones si el cliente no tiene un bloque de manejo de errores configurado.
- B) Porque el método POST requiere obligatoriamente una excepción para poder enviar datos JSON.
- C) Porque si el controlador de red (como APIC) está fuera de línea, el script fallará y detendrá toda la automatización sin informar el error.

Escenario de 'Saber Hacer': Depuración de Rutas

Un colega ha escrito la siguiente ruta para eliminar un dispositivo, pero no funciona como se esperaba:

```
@app.route('/devices/<id>', methods=['DELETE'])
def delete_device(id):
    # Lógica para borrar...
```

Si los IDs en tu lista de Python son números enteros, explica qué cambio es necesario en la definición de la ruta y por qué es importante el tipado en las URIs de una REST API.

```
=====
=====
=====
=====
=====
=====
=====
=====
```

Clave de Respuestas Técnica

Esta guía proporciona las soluciones detalladas, el razonamiento técnico y los fragmentos de código resueltos para las actividades de la Unidad II: Introducción a REST API.

Actividad 1: Fundamentos de APIs y Estructuras JSON en Redes

1. Identificación de Pares (Nivel Superior):

- `"hostname" : "R1-Core"`
- `"uptime_days" : 45`
- `"managed_by_apic" : true`
- `"interfaces" : [arreglo de objetos]`

2. Tipos de Datos:

- **String:** "R1-Core" o "GigabitEthernet0/0"
- **Number:** 45 o 1500
- **Boolean:** true o false
- **Null:** null (en el campo description de Gi0/1)

3. Detección de Errores de Sintaxis:

- **C) Existe una coma final (trailing comma) después del valor de 'description'.**
- **D) Faltan comas para separar los pares clave-valor después de 'vlan_id' y 'name'.**
- *Nota técnica:* JSON es estricto; a diferencia de los diccionarios de Python, no permite comas tras el último elemento de un objeto o arreglo.

4. Reflexión Técnica:

- **Respuesta sugerida:** Es crítico porque los controladores actúan como la "Fuente de Verdad" de la red. Un JSON mal formado puede causar que el motor de validación del controlador rechace la petición, o peor aún, si el error es de lógica y no de sintaxis, podría desconfigurar servicios críticos. La validación asegura la integridad de la automatización.

Actividad 2: Análisis de Datos de Red con Python

1. Script de Python (Filtrado de Hostnames):

```
python
import json

inventario = json.loads(inventario_json)
for dispositivo in inventario:
    if dispositivo["status"] == "down":
        print(dispositivo["hostname"])
```

- *Resultado esperado:* Imprimirá `Dist-SW02` y `Edge-R01`.
- *Error común:* Intentar acceder a `inventario_json["status"]` directamente sin usar `json.loads()`. Recuerde que el JSON inicial es un String, no un objeto iterable.

2. Mapa de Árbol (JSON vs XML):

- **JSON:** Pares clave-valor, Ligero, Tipos nativos (bool, int), Ideal para REST.
- **XML:** Etiquetas de apertura/cierre, Verboso (pesado), Todo es texto, Usado en SOAP/NETCONF.

3. Diferencia fundamental:

- **A. JSON tiene tipos de datos nativos (números, booleanos), mientras que en XML todo se trata inicialmente como texto.**

4. Justificación Técnica:

- **Respuesta sugerida:** JSON reduce el consumo de ancho de banda y la carga de procesamiento (CPU) del controlador al tener menos caracteres (sin etiquetas de cierre), lo que permite gestionar miles de dispositivos con menor latencia.

Actividad 3: Interacción con REST APIs y Protocolo HTTP

Parte 1: Métodos HTTP

1. Lista de VLANs: **GET**
2. Nuevo objeto de red: **POST**
3. Reemplazar configuración OSPF: **PUT**
4. Eliminar ruta estática: **DELETE**
5. Modificar descripción (parcial): **PATCH**

Parte 2: Códigos de Estado

1. Interfaz inexistente: **B. 404 Not Found**
2. Token inválido/expirado: **A. 401 Unauthorized**

Parte 3: Diagnóstico y Justificación

- **3. Razones para 400 Bad Request:** 1) Sintaxis JSON incorrecta (falta una coma o comilla). 2) Datos fuera de rango (ej. una IP inválida como 999.999.999.999).
- **4. POST vs PUT (Idempotencia):** `PUT` es idempotente; si se envía la misma petición 10 veces, el resultado final es el mismo (la configuración queda igual). `POST` no suele serlo; enviar 10 veces la misma petición podría crear 10 recursos duplicados.

Actividad 4: Automatización de Pruebas en Postman

1. Resultado de tiempo de respuesta:

- **A. La prueba fallará (FAIL) y mostrará un error de aserción.**

2. Script de tiempo (< 200ms):

```
javascript
pm.test("Tiempo de respuesta menor a 200ms", () => {
  pm.expect(pm.response.responseTime).to.be.below(200);
});
```

3. Script de validación de campo:

```
javascript
pm.test("Contiene hostname", () => {
  const data = pm.response.json();
  pm.expect(data).to.have.property('hostname');
});
```

4. Justificación:

- **Respuesta sugerida:** El código 200 solo indica que el servidor respondió, pero no garantiza que los datos sean correctos. Validar el JSON asegura que la configuración aplicada es la que realmente deseamos y que el dispositivo devolvió la información esperada.